

Data Structures

(CS2001)

Date: Nov 2025

Note: Solve on the question Paper

CLO # 2: Evaluate different data structures in terms of memory complexity and time

Q. No 1: (Marks: 5)

Course Instructor(s)

Zareen, Waqas,

Uzair, Naveed, Arooj, _____
Roll No _____ Section _____

Razi, and Faisal

#include
<stack>

Student Signature

Midterm 2 Exam

Total Time (Hrs.): 1

Total Marks: 25

Total Questions: 3

```
template<typename T>
void descendIntoAbyss(stack<T> &s, T x) {
    if (s.empty()) {
        s.push(x);
        return;
    }
    T topElem = s.top();
    s.pop();
    descendIntoAbyss(s, x);
    s.push(topElem);
}
```

```
template<typename T>
void invertChronology(stack<T> &s) {
    if (s.empty()) return;
    T topElem = s.top();
    s.pop();
    invertChronology(s);
    descendIntoAbyss(s, topElem);
}
```

```
template<typename T>
void Display(stack<T> s) {
    while (!s.empty()) {
        cout << s.top() << " ";
        s.pop();
    }
}
```

```
int main() {
    stack<int> s;
    for(int i=1;i<=10;i++)
        s.push(i*3);
    cout << "Before:\n";
    Display(s);
    invertChronology(s);
    cout << "\nAfter:\n";
    Display(s);
    return 0;
}
```

Output: Solution

Before:

Stack (top → bottom): 30 27 24 21 18 15 12 9 6 3

After:

Stack (top → bottom): 3 6 9 12 15 18 21 24 27 30

CLO # 1: Demonstrate basic concepts of data structure and algorithms

Q. No 1:

(Marks: 8+2)

Write a recursive function **maxDataPath** that finds the maximum sum of data values along any path in the tree (**path may start and end at any node, not necessarily the root**).

Each node can be part of the path at most once. The path may go through the root or be entirely in a subtree.

Example Input (Tree): <pre> 10 / \ 2 10 / \ \ 20 1 -25 / \ 3 4 </pre>	Expected Output: Maximum Data Path Sum = 42 Explanation: The path with maximum sum is 20 → 2 → 10 → 10, giving 42.
<pre> struct BNode { int data; BNode *left, *right; }; </pre>	Prototype of the required function int Btree :: maxDataPath(BNode* node, int & Max)

Write **recursive C++ code** and provide comments explaining each step. Also, mention the **time complexity of your algorithm and briefly explain it**. Clearly show the base case and recursive case in your function.

Hint: Each recursive call should return the maximum sum of a path starting at the current node.

Note: No global/static variables allowed

#include <iostream>

using namespace std;

```

int maxSum(Node* root, int &globalMax) {
    if (root == NULL) return 0;
    // Compute max path sum in left and right subtrees
    int left = maxSum(root->left, globalMax);
    int right = maxSum(root->right, globalMax);
    // Max single path through current node
    int single = root->data + (left > right ? left : right);
    if (single < root->data) single = root->data;

    // Max path passing through current node (including both sides)

```

National University of Computer and Emerging Sciences

Lahore Campus

```
int fullPath = left + right + root->data;
```

```
if (fullPath > globalMax) globalMax = fullPath;
```

```
if (single > globalMax) globalMax = single;
```

```
return single;
```

```
}
```

National University of Computer and Emerging Sciences

Lahore Campus

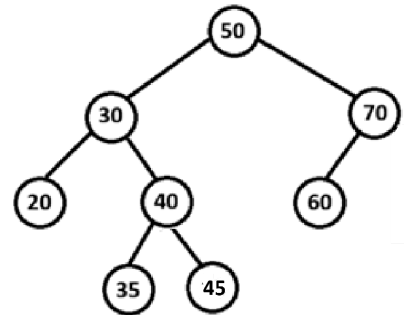
CLO #2: Evaluate different data structures in terms of memory complexity and time

Question 2: [Marks: 5+5] Consider the AVL tree given below

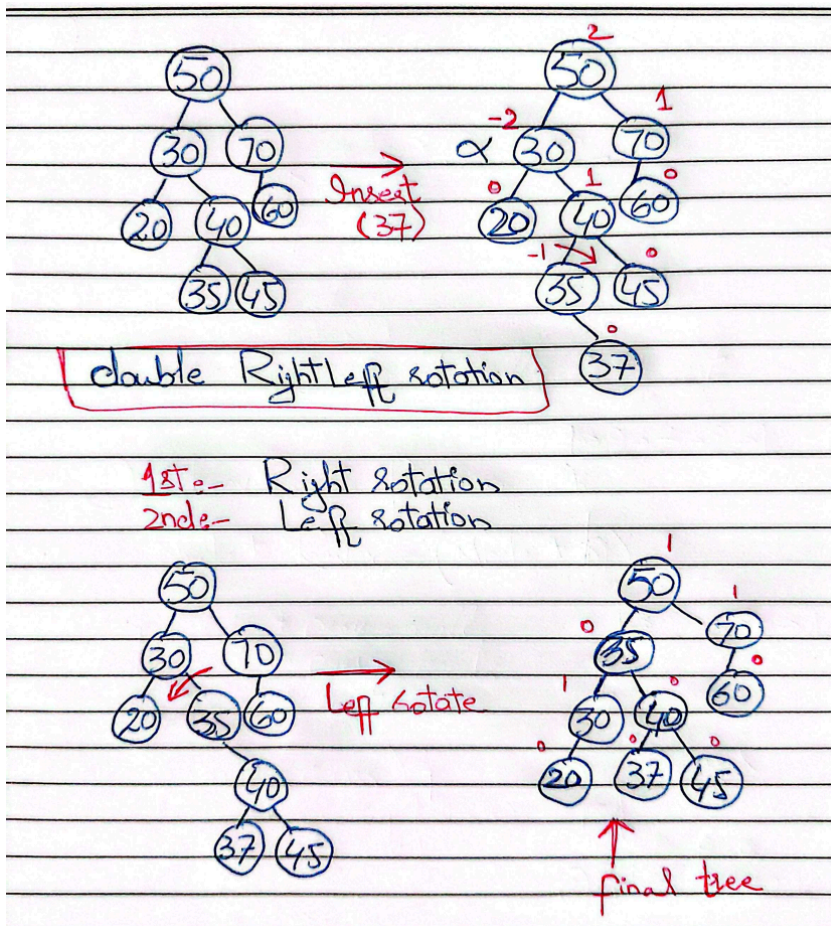
Perform the following operations on the given tree. The operations are not in sequence and should be performed on the given AVL tree.

Draw the tree after each rotation operation.

Mention the type of rotation(s) performed to maintain AVL property. Show the final balanced AVL tree after operations.



a. Insert 37



b. Delete 70 (perform this operation on the given tree)

